Relational Separation Logic for Compiler Verification
Research internship proposal, M2

Xavier Leroy, Collège de France, xavier.leroy@college-de-france.fr
François Pottier, Inria Paris, francois.pottier@inria.fr

2026

1 Overview

A recurring problem is to prove that two programs A and B are equivalent (that is, they have exactly the same observable behaviors) or that a program B refines a program A (that is, every observable behavior of B is a possible observable behavior of A).

This problem arises when we programmers progressively improve the performance of a program. Here, A is the original, naive, unoptimized code, and B is the hand-optimized, more efficient code. We expect B to refine A.

The problem also arises in the formal verification of compilers. Each pass of a compiler is a program transformation that optimizes its input code (in the hope of improving its efficiency) and/or translates it to a lower-level language. For the compiler to be correct (semantics-preserving), the output $F(A)$ of a compilation pass F must refine its input A. This property must hold for all possible inputs A.

To show an equivalence or refinement between two programs, several proof techniques are known. Looking at compiler verification projects, CompCert uses simulation diagrams: each execution step of one program is put in correspondence with zero, one or more execution steps of the other program, while preserving an invariant that relates the states of both programs. CakeML and its extension Pure CakeML use a combination of simulation diagrams and of equational reasoning in an equational theory for a pure functional language. Both approaches are unsatisfactory: equational theories are often incomplete, and simulation diagrams become extremely complex when the control flow structures of the two programs differ significantly (for example, after two nested loops are exchanged).

Proofs of program refinement can also be conducted using relational program logics. A non-relational program logic offers rules to deduce facts of the form $\{P\}\ c\ \{Q\}$ about the executions of a code fragment c with precondition P and postcondition Q. Similarly, a relational program logic offers rules to deduce facts of the form $\{P\}\ c_1 \preceq c_2\ \{Q\}$ about the executions of two code fragments c_1 and c_2, with a precondition P that relates the initial states of these executions, and a postcondition Q that relates their final states. A relational Separation Logic is similar, but uses the techniques of separation logic to better support local reasoning about mutable store and about concurrency.

The purpose of this research internship is to investigate the usability of relational separation logics for compiler verification. Can one express semantic preservation for a compilation pass F as a relational triple $\{P\}\ F(c) \preceq c\ \{Q\}$? What are the appropriate (relational) pre- and postconditions P and Q? Which logical rules are needed in order to verify classic compilation passes, as found in CompCert or CakeML? How can one take advantage of Separation Logic predicates and of local reasoning in compiler verification proofs?

2 Roadmap

We intend to first explore (manual) proofs of equivalence between two (fixed) programs before we attack proofs of compiler correctness.

We intend to first explore a tiny programming language before studying richer languages. This will allow us to focus on control structures (loops, jumps, perhaps functions) and/or on accesses to memory (the stack, the heap).

We intend to first explore the situation where the two programs are written in the same language. At a later point, we would like to also study the situation where the source and target program are written in different languages and/or have different views of memory.

We may wish to first study and understand relational Hoare logics before investigating relational Separation Logics.

It is worth noting that ordinary (unary) program logics are usually used to verify a program, that is, to prove that it is correct in some sense. Here, we envision a different use case: a compiler must be able to compile unverified programs, so, when the compiler runs, we have no idea whether (or in what sense) the program is correct. We want to use a relational (binary) program logic to prove that the compiled program and the original program are equivalent, but we cannot prove that these programs are "correct".

Examples of interesting program transformations include function inlining and function specialization (e.g., specializing a possibly-recursive function for an argument whose shape is partially statically known). Replacing a call to `iter` with a loop might be viewed as an example of function specialization.

3 Prerequisites

A solid programming background, including fluency in OCaml, is highly desirable. Familiarity with the operational semantics of programming languages (MPRI 2.4) and with Hoare logic and Separation Logic (MPRI 2.36.1) is essential. Familiarity with Rocq or with another proof assistant (MPRI 2.7.1 and 2.7.2) is essential.

4 Practical details

This internship will be jointly supervised by Xavier Leroy (Collège de France & Inria Paris) and François Pottier (Inria Paris). It will take place at Inria Paris from March 2026 to August 2026 approximately.

5 Reading list

To learn about Separation Logic for sequential programs, we suggest reading the educational pearl by Charguéraud (2020), as well as the associated course material (Charguéraud, 2021). Learning about the logical foundations of Iris (Jung, Krebbers, Jourdan, Bizjak, Birkedal, and Dreyer, 2018) is necessary. The surveys by O'Hearn (2019) and by Brookes and O'Hearn (2016) provide broader information on Separation Logic and its many variants.

According to Naumann (2020), Relational Hoare Logic was first studied by Francez (1983). The term "Relational Hoare Logic" itself seems to have been coined by Benton (2004), who applied this logic to proofs of compiler correctness. It was soon transported to the setting of Relational Separation Logic by Yang (2007). A modern variant of Relational Separation Logic, based on Iris, is ReLoC (Frumin, Krebbers, and Birkedal, 2018, 2021). ReLoC has been used to establish refinements between concurrent programs: for example, it can be used to show that a fine-grained (lock-free)

concurrent data structure refines a simpler coarse-grained (lock-based) data structure. Simuliris (Gäher, Sammler, Spies, Jung, Dang, Krebbers, Kang, and Dreyer, 2022), another Iris-based logic, is similar in some ways to ReLoC, but uses co-induction instead of Iris's guarded recursion, and is able to establish termination-preserving refinements. Recent work of interest includes Refinement Composition Logic (Song and Lee, 2024), Conditional Contextual Refinement (Song et al., 2023; Song and Cho, 2025), and Lilo (Lee et al., 2025). The literature on relational program logics also includes work by Crespo and Kunz (2011), by Aguirre, Barthe, Gaboardi, Garg, and Strub (2019), by Nagasamudram, Banerjee, and Naumann (2023a,b), by Antonopoulos, Koskinen, Le, Nagasamudram, Naumann, and Ngo (2023), by Dardinier and Müller (2024), and by Bonneau (2025).

Beacuse co-induction plays a central role in simulation arguments, and can be difficult to use in practice, it is worth understanding it in depth. Recommended reading includes the papers by Pous (2016); Schäfer and Smolka (2017); Pous and Sangiorgi (2019). Two powerful Rocq libraries for co-induction are Paco[1] (Hur, Neis, Dreyer, and Vafeiadis, 2013) and Damien Pous's `coinduction` library.[2] Zakowski, He, Hur, and Zdancewic (2020) discuss co-inductive proofs of (weak) bisimilarity. Correnson et al. recent work may be relevant (Correnson and Finkbeiner, 2025; Correnson et al., 2025).

The CompCert compiler (Leroy, 2009a,b) uses traditional simulation arguments. When the source program and the transformed program have different views of memory, these arguments can involve "memory injections". The treatment of stuttering in simulations is often a subtle and problematic topic; Cho, Song, Lee, Gäher, and Dreyer (2023) offer an interesting point of view on this topic. Chappe's unpublished paper The Sims may be relevant, and includes an application to CompCert.

Work that applies Relational Separation Logic to the verification of program transformations includes a proof of compiler correctness for a language with session types (Tassarotti, Jung, and Harper, 2017), proofs of some reorderings of memory accesses in concurrent programs (Gäher et al., 2022), and Allain et al.'s verification of the "tail modulo cons" transformation (Allain, Bour, Clément, Pottier, and Scherer, 2025).

References

Alejandro Aguirre, Gilles Barthe, Marco Gaboardi, Deepak Garg, and Pierre-Yves Strub. A relational logic for higher-order programs. *Journal of Functional Programming*, 29:e16, 2019. URL https://arxiv.org/abs/1703.05042.

Clément Allain, Frédéric Bour, Basile Clément, François Pottier, and Gabriel Scherer. Tail modulo cons, OCaml, and relational separation logic. *Proceedings of the ACM on Programming Languages*, 9(POPL):2337–2363, January 2025. URL http://cambium.inria.fr/~fpottier/publis/tmc-popl2025.pdf.

Timos Antonopoulos, Eric Koskinen, Ton Chanh Le, Ramana Nagasamudram, David A. Naumann, and Minh Ngo. An algebra of alignment for relational verification. *Proceedings of the ACM on Programming Languages*, 7(POPL):573–603, 2023. URL https://doi.org/10.1145/3571213.

Nick Benton. Simple relational correctness proofs for static analyses and program transformations. In *Principles of Programming Languages (POPL)*, pages 14–25, January 2004. URL https://www.microsoft.com/en-us/research/publication/simple-relational-correctness-proofs-for-static-analyses-and-program-transformations/.

Benjamin Bonneau. Relational reasoning on monadic semantics. In *Journées Françaises des Langages Applicatifs (JFLA)*, January 2025. URL https://hal.science/hal-04859423.

[1] https://plv.mpi-sws.org/paco/

[2] https://github.com/damien-pous/coinduction

Stephen Brookes and Peter W. O'Hearn. Concurrent separation logic. *SIGLOG News*, 3(3):47–65, 2016. URL http://siglog.hosting.acm.org/wp-content/uploads/2016/07/siglog_news_9.pdf.

Arthur Charguéraud. Separation logic for sequential programs (functional pearl). *Proceedings of the ACM on Programming Languages*, 4(ICFP):116:1–116:34, 2020. URL https://doi.org/10.1145/3408998.

Arthur Charguéraud. Separation logic foundations. In Benjamin C. Pierce, editor, *Software Foundations*, volume 6. 2021. URL https://softwarefoundations.cis.upenn.edu/slf-current/index.html.

Minki Cho, Youngju Song, Dongjae Lee, Lennard Gäher, and Derek Dreyer. Stuttering for free. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA2):1677–1704, 2023. URL https://doi.org/10.1145/3622857.

Arthur Correnson and Bernd Finkbeiner. Coinductive proofs for temporal hyperliveness. *Proceedings of the ACM on Programming Languages*, 9(POPL):1568–1595, January 2025. URL https://doi.org/10.1145/3704889.

Arthur Correnson, Iona Kuhn, and Bernd Finkbeiner. Almost fair simulations. *Proceedings of the ACM on Programming Languages*, 9(ICFP):222–245, August 2025. URL https://doi.org/10.1145/3747512.

Juan Manuel Crespo and César Kunz. A machine-checked framework for relational separation logic. In *Software Engineering and Formal Methods*, volume 7041 of *Lecture Notes in Computer Science*, pages 122–137. Springer, November 2011. URL https://software.imdea.org/~ckunz/rsl2sl/SEFM2011.pdf.

Thibault Dardinier and Peter Müller. Hyper Hoare logic: (dis-)Proving program hyperproperties. *Proceedings of the ACM on Programming Languages*, 8(PLDI):1485–1509, 2024. URL https://doi.org/10.1145/3656437.

Nissim Francez. Product properties and their direct verification. *Acta Informatica*, 20:329–344, 1983. URL https://doi.org/10.1007/BF00264278.

Dan Frumin, Robbert Krebbers, and Lars Birkedal. ReLoC: A mechanised relational logic for fine-grained concurrency. In *Logic in Computer Science (LICS)*, pages 442–451, July 2018. URL https://iris-project.org/pdfs/2018-lics-reloc-final.pdf.

Dan Frumin, Robbert Krebbers, and Lars Birkedal. ReLoC reloaded: A mechanized relational logic for fine-grained concurrency and logical atomicity. *Logical Methods in Computer Science*, 17(3), 2021. URL https://arxiv.org/abs/2006.13635v3.

Lennard Gäher, Michael Sammler, Simon Spies, Ralf Jung, Hoang-Hai Dang, Robbert Krebbers, Jeehoon Kang, and Derek Dreyer. Simuliris: a separation logic framework for verifying concurrent program optimizations. *Proceedings of the ACM on Programming Languages*, 6(POPL):1–31, 2022. URL https://doi.org/10.1145/3498689.

Chung-Kil Hur, Georg Neis, Derek Dreyer, and Viktor Vafeiadis. The power of parameterization in coinductive proof. In *Principles of Programming Languages (POPL)*, pages 193–206, January 2013. URL https://plv.mpi-sws.org/paco/ppcp.pdf.

Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018. URL https://people.mpi-sws.org/~dreyer/papers/iris-ground-up/paper.pdf.

Dongjae Lee, Janggun Lee, Taeyoung Yoon, Minki Cho, Jeehoon Kang, and Chung-Kil Hur. Lilo: A higher-order, relational concurrent separation logic for liveness. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA1):1267–1294, April 2025. URL https://doi.org/10.1145/3720525.

Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7): 107–115, 2009a. URL https://doi.org/10.1145/1538788.1538814.

Xavier Leroy. A formally verified compiler back-end. *Journal of Automated Reasoning*, 43(4):363–446, 2009b. URL https://xavierleroy.org/publi/compcert-backend.pdf.

Ramana Nagasamudram, Anindya Banerjee, and David A. Naumann. Alignment complete relational Hoare logics for some and all. Preprint, 2023a. URL https://arxiv.org/pdf/2307.10045.

Ramana Nagasamudram, Anindya Banerjee, and David A. Naumann. The WhyRel prototype for modular relational verification of pointer programs. In *Tools and Algorithms for Construction and Analysis of Systems (TACAS)*, volume 13994 of *Lecture Notes in Computer Science*, pages 133–151. Springer, April 2023b. URL https://arxiv.org/abs/2303.14314.

David A. Naumann. Thirty-seven years of relational hoare logic: Remarks on its principles and history. In *International Symposium on Leveraging Applications of Formal Methods*, volume 12477 of *Lecture Notes in Computer Science*, pages 93–116. Springer, October 2020. URL https://arxiv.org/abs/2007.06421.

Peter W. O'Hearn. Separation logic. *Communications of the ACM*, 62(2):86–95, 2019. URL https://doi.org/10.1145/3211968.

Damien Pous. Coinduction all the way up. In *Logic in Computer Science (LICS)*, pages 307–316, July 2016. URL https://doi.org/10.1145/2933575.2934564.

Damien Pous and Davide Sangiorgi. Bisimulation and coinduction enhancements: A historical perspective. *Formal Aspects of Computing*, 31(6):733–749, 2019. URL https://doi.org/10.1007/s00165-019-00497-w.

Steven Schäfer and Gert Smolka. Tower induction and up-to techniques for CCS with fixed points. In *Relational and Algebraic Methods in Computer Science*, volume 10226 of *Lecture Notes in Computer Science*, pages 274–289, May 2017. URL https://www.ps.uni-saarland.de/Publications/documents/SchaeferSmolka_2017_Tower.pdf.

Youngju Song and Minki Cho. Ccr 2.0: High-level reasoning for conditional refinements. Unpublished, July 2025. URL https://arxiv.org/abs/2507.04298.

Youngju Song and Dongjae Lee. Refinement composition logic. *Proceedings of the ACM on Programming Languages*, 8(ICFP):573–601, August 2024. URL https://doi.org/10.1145/3674645.

Youngju Song, Minki Cho, Dongjae Lee, Chung-Kil Hur, Michael Sammler, and Derek Dreyer. Conditional contextual refinement. *Proceedings of the ACM on Programming Languages*, 7(POPL): 1121–1151, 2023. URL https://doi.org/10.1145/3571232.

Joseph Tassarotti, Ralf Jung, and Robert Harper. A higher-order logic for concurrent termination-preserving refinement. In *European Symposium on Programming (ESOP)*, volume 10201 of *Lecture Notes in Computer Science*, pages 909–936. Springer, April 2017. URL https://iris-project.org/pdfs/2017-esop-refinement-final.pdf.

Hongseok Yang. Relational separation logic. *Theoretical Computer Science*, 375(1-3):308–334, 2007. URL https://doi.org/10.1016/j.tcs.2006.12.036.

Yannick Zakowski, Paul He, Chung-Kil Hur, and Steve Zdancewic. An equational theory for weak bisimulation via generalized parameterized coinduction. In *Certified Programs and Proofs (CPP)*, pages 71–84, January 2020. URL https://doi.org/10.1145/3372885.3373813.